

Native VPS Deployment Guide (Ubuntu/Debian)

This guide covers how to deploy the Webmap application directly on a Linux VPS without Docker, configuring everything natively.

Prerequisites

- A VPS running Ubuntu 22.04 or Debian 12.
 - A registered domain name (e.g., webmap.rhdbridge.com) pointed to your VPS IP address.
-

1. System Setup and Dependencies

Update your system and install the required tools:

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y python3 python3-venv python3-pip nginx certbot python3-
certbot-nginx
```

Install PostgreSQL and PostGIS:

```
sudo apt install -y postgresql postgresql-contrib postgis postgresql-14-
postgis-3
```

2. Database Configuration

Log in to the PostgreSQL prompt:

```
sudo -u postgres psql
```

Run the following SQL commands to create your database, user, and enable PostGIS. (Note: Replace *<webmap_db>*, *<webmap_user>*, and *<your_secure_password>* with your actual desired names/passwords):

```
CREATE DATABASE <webmap_db>;
CREATE USER <webmap_user> WITH ENCRYPTED PASSWORD '<your_secure_password>';
GRANT ALL PRIVILEGES ON DATABASE <webmap_db> TO <webmap_user>;
\c <webmap_db>
CREATE EXTENSION postgis;
\q
```

3. Application Setup

Clone the repository into `/var/www/`:

```
cd /var/www/  
sudo git clone https://your-repo-url/webmap.git  
sudo chown -R $USER:$USER /var/www/webmap  
cd webmap
```

Create a Python virtual environment and install dependencies:

```
python3 -m venv venv  
source venv/bin/activate  
pip install -r requirements.txt
```

Create your `.env` file for the backend:

```
nano .env
```

Paste your secrets (matching the database credentials you set above):

```
WEBMASTER_USERNAME=<admin_name>  
WEBMASTER_PASSWORD=<super_secret_password>  
POSTGRES_USER=<webmap_user>  
POSTGRES_PASSWORD=<your_secure_password>  
POSTGRES_DB=<webmap_db>  
POSTGRES_HOST=localhost  
POSTGRES_PORT=5432
```

4. Run the Backend with Systemd

Create a systemd service file to keep the FastAPI backend running forever:

```
sudo nano /etc/systemd/system/webmap-backend.service
```

Add the following configuration:

```
[Unit]  
Description=Webmap FastAPI Backend  
After=network.target postgresql.service
```

```
[Service]
User=root
WorkingDirectory=/var/www/webmap
Environment="PATH=/var/www/webmap/venv/bin"
ExecStart=/var/www/webmap/venv/bin/uvicorn backend.main:app --host
127.0.0.1 --port 8484
Restart=always

[Install]
WantedBy=multi-user.target
```

Start and enable the service:

```
sudo systemctl daemon-reload
sudo systemctl start webmap-backend
sudo systemctl enable webmap-backend
```

5. Configure Nginx (HTTP Protocol)

First, we will set up the basic Nginx configuration to serve the application over standard HTTP (Port 80).

Create an Nginx server block:

```
sudo nano /etc/nginx/sites-available/webmap
```

Paste the following HTTP configuration:

```
server {
    listen 80;
    server_name webmap.rhdbridge.com;

    # Serve Frontend static files
    location / {
        root /var/www/webmap/frontend;
        index index.html;
        try_files $uri $uri/ /index.html;
    }

    # Reverse proxy for FastAPI Backend
    location /api/ {
        proxy_pass http://127.0.0.1:8484;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

```
}  
}
```

Enable the site and test the configuration:

```
sudo ln -s /etc/nginx/sites-available/webmap /etc/nginx/sites-enabled/  
sudo nginx -t  
sudo systemctl restart nginx
```

At this point, your site is live via <http://webmap.rhdbridge.com>.

6. Secure with SSL (HTTPS Protocol)

To encrypt traffic over HTTPS (Port 443), you have two options:

Method A: Automated SSL with Certbot (Recommended)

If you want a free Let's Encrypt certificate, run:

```
sudo certbot --nginx -d webmap.rhdbridge.com
```

Certbot will automatically modify your Nginx configuration to listen on port 443, attach the certificates, and forcefully redirect all HTTP traffic to HTTPS.

Method B: Manual SSL Configuration

If your domain manager provides custom SSL certificates ([.crt](#) and [.key](#)), upload them to your server (e.g., [/etc/ssl/certs/](#) and [/etc/ssl/private/](#)). Then, modify your Nginx block ([/etc/nginx/sites-available/webmap](#)) to handle HTTPS manually:

```
# Redirect HTTP to HTTPS  
server {  
    listen 80;  
    server_name webmap.rhdbridge.com;  
    return 301 https://$host$request_uri;  
}  
  
# Serve HTTPS  
server {  
    listen 443 ssl;  
    server_name webmap.rhdbridge.com;  
  
    ssl_certificate /etc/ssl/certs/your_domain.crt;  
    ssl_certificate_key /etc/ssl/private/your_domain.key;  
  
    # Recommended SSL settings
```

```
ssl_protocols TLSv1.2 TLSv1.3;
ssl_ciphers HIGH:!aNULL:!MD5;

location / {
    root /var/www/webmap/frontend;
    index index.html;
    try_files $uri $uri/ /index.html;
}

location /api/ {
    proxy_pass http://127.0.0.1:8484;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto https;
}
}
```

Restart Nginx (`sudo systemctl restart nginx`) to apply. Your webmap is now securely live natively on your VPS!